

Diseño de una Línea de Productos de Software para un SISTEMA de GESTIÓN CLÍNICA SAAS en ECUADOR

Balarezo. L., Guido C., Gonzales L.

Carrera de Software, Universidad Técnica del Norte

Fábrica de Software – Ph.D. Antonio Quiña-Mera

Fecha de entrega: 11 mayo 2026

lfgonzalez@utn.edu.ec , gacalderon@utn.edu.ec

Resumen—Este artículo presenta el diseño de una Línea de Productos de Software (LPS) para una plataforma SaaS de gestión clínica orientada al mercado ecuatoriano. Se seleccionó el dominio de e-health por su alta variabilidad (especialidades médicas, regulaciones locales, métodos de pago) y la necesidad de reducir el tiempo de desarrollo frente al enfoque tradicional “clone & own”. Se construyó un Feature Model FODA con 27 características, 5 restricciones cruzadas y 456 configuraciones válidas. Se definió una arquitectura de referencia C4 Level 2 con mecanismos de variación (Strategy, Dependency Injection, Feature Flags) y se derivaron tres productos concretos: MediBase (básico), OdontoPro (dental) y CardioPremium (cardiología con telemedicina). El análisis económico basado en Clements & Northrop (2002) muestra un punto de equilibrio en el cuarto producto, con un ROI esperado del 350% a tres años.

Palabras clave— Línea de Productos de Software, Feature Model, FODA, FeatureIDE, e-health, variabilidad, arquitectura de referencia, Configuration Knowledge, ROI.

I. 1. Introducción

La industria del software médico en Ecuador enfrenta un desafío creciente: decenas de clínicas y consultorios requieren sistemas de gestión digital, pero cada uno demanda adaptaciones por especialidad (odontología, cardiología, medicina general), tamaño de la clínica, métodos de pago locales (Kushki, PayPhone) y normativas de protección de datos (LOPD). El enfoque tradicional de desarrollar cada sistema desde cero o mediante “clone & own” resulta insostenible: los costos marginales no disminuyen, la deuda técnica se acumula y la calidad varía entre productos.

Una Línea de Productos de Software (LPS) ofrece una alternativa sistemática: construir una plataforma base (core assets) con variabilidad explícita, de la cual se derivan productos concretos mediante configuración. Este trabajo aplica la metodología FODA (Feature-Oriented Domain Analysis) y la herramienta FeatureIDE para modelar una LPS de gestión clínica SaaS, siguiendo los lineamientos del proyecto integrador de la asignatura Fábrica de Software.

El artículo se organiza como sigue. La Sección 2 justifica el dominio seleccionado. La Sección 3 presenta el análisis de comunalidad y variabilidad. La Sección 4 describe el feature model completo. La Sección 5 muestra la arquitectura de referencia. La Sección 6 detalla las tres configuraciones de producto. La Sección 7 expone el configuration knowledge. La Sección 8 analiza el retorno de inversión (ROI). Finalmente, la Sección 9 concluye con lecciones aprendidas.

II. 2. Selección del Dominio

A. 2.1 Dominio elegido: SaaS para Gestión de Clínicas Médicas (E-Health)

El dominio seleccionado es una plataforma SaaS multi-tenant para la gestión integral de consultorios y clínicas privadas en Ecuador. Los productos de esta línea cubren desde pequeños consultorios de medicina general hasta centros especializados en odontología o cardiología.

B. 2.2 Justificación

El dominio e-health presenta una alta variabilidad natural que lo hace idóneo para una LPS:

Dimensión de variabilidad	Rango de variación
Especialidad médica	Medicina general, odontología, cardiología

Tamaño de la clínica	1-2 médicos (básico) hasta 10+ especialistas (premium)
Métodos de pago	Efectivo (cobro manual), Kushki, PayPhone, DeUna!
Regulaciones locales	LOPD (Ecuador), facturación electrónica SRI
Módulos opcionales	Telemedicina, receta electrónica, notificaciones WhatsApp

El enfoque ad-hoc requeriría desarrollar al menos 15 sistemas distintos con un alto porcentaje de código duplicado. Una LPS permite reutilizar la lógica central (pacientes, agenda, facturación base) mientras se varían solo los módulos diferenciales. Este enfoque reduce el time-to-market y garantiza la calidad mediante el testing único de los core assets.

C. 2.3 Portafolio de productos esperado

El portafolio inicial incluye tres productos concretos (ver Sección 6), pero el modelo permite derivar hasta 456 configuraciones válidas, cubriendo la mayoría de los escenarios del mercado ecuatoriano.

III. 3. Análisis de Comunalidad y Variabilidad (CVA)

La Tabla I resume las características comunes y variables identificadas mediante entrevistas con stakeholders (médicos, administradores de clínicas) y análisis de sistemas competidores.

1) Tabla I

Análisis de Comunalidad y Variabilidad (CVA) – 8 características comunes y 6 puntos de variación

Feature	Tipo	Descripción	Productos que la incluyen
Autenticación	Común	Login con usuario/contraseña y recuperación	100%
Control de Roles	Común	Permisos diferenciados (médico, recepción, admin)	100%
Auditoría de Datos	Común	Registro de accesos a historia	100%

			clínica (cumplimiento LOPD)	
Gestión de Pacientes	Común		CRUD de pacientes, datos de contacto, alergias	100%
Agenda de Citas	Común		Calendario visual, asignación de turnos	100%
Historia Clínica Base	Común		Signos vitales, motivo de consulta, texto libre	100%
Cobro Manual	Común		Facturación en efectivo o transferencia bancaria	100%
Catálogo de Servicios Especialidad	Común	Variable (OR)	Lista de precios configurable	100%
			Rama médica de la clínica	Medicina general, odontología, cardiología, Kushki, PayPhone, DeUna!
Métodos de Pago Digital	Variable (XOR)		Pasarelas online	Solo productos premium
Telemedicina	Variable (Optional)		Videollamada integrada para consultas remotas	Productos intermedios y premium
Receta Electrónica	Variable (Optional)		Envío digital de prescripciones con firma	

Notificaciones	Variable (OR)	Alertas a pacientes por Email o WhatsApp	Productos con plan de marketing
Imágenes Médicas	Variable (Optional)	Almacenamiento de radiografías (DICOM)	Solo cardiología y odontología avanzada

Justificación de negocio: El 100% de los sistemas de gestión clínica requieren autenticación y control de roles por ley de protección de datos. La especialidad determina qué módulos clínicos se activan (odontograma para odontología, soporte DICOM para cardiología). Los pagos digitales son obligatorios si se incluye telemedicina, ya que no es posible cobrar manualmente en una consulta remota.

IV. 4. Feature Model (FODA)

A. 4.1 Modelo completo

El feature model fue construido en FeatureIDE (Eclipse plugin) siguiendo la notación FODA. La Figura 1 muestra el diagrama completo, que totaliza 27 características (superando el mínimo de 20 requerido) con los siguientes tipos:

Mandatory (círculo relleno): Autenticación, Control Roles, Auditoría, Gestión de Pacientes, Agenda, Historia Clínica Base, Cobro Manual, etc.

Optional (círculo vacío): Telemedicina, Receta Electrónica, Imágenes Médicas, Notificaciones, Pagos Online.

Alternative (XOR – arco relleno): Pasarelas de pago (Kushki, PayPhone, DeUna!). Selección exactamente una.

Or-Group (arco vacío): Especialidad (medicina general, odontología, cardiología – al menos una) y Notificaciones (Email, WhatsApp – una o varias).

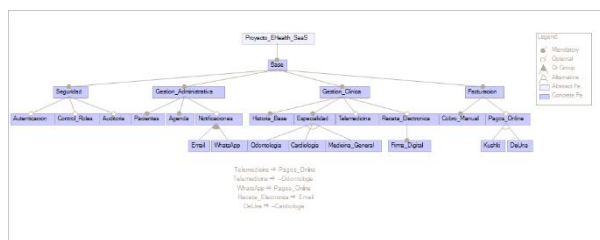


Figura 1
Diagrama de características (Feature Model) de la LPS de Gestión Clínica SaaS (Incluir captura de FeatureIDE mostrando el árbol completo con: raíz "EHealth_System", ramas

Seguridad, Gestión Administrativa, Gestión Clínica, Facturación; subramas con los tipos indicados)

4.2 Lista completa de características

EHealth_System (Raíz)

- └─ Seguridad (M)
 - └─ Autenticación (M)
 - └─ Control Roles (M)
 - └─ Auditoría (M)
- └─ Gestión Administrativa (M)
 - └─ Pacientes (M)
 - └─ Agenda (M)
 - └─ Notificaciones (O) – [OR: Email,

WhatsApp]

- └─ Gestión Clínica (M)
 - └─ Historia Clínica Base (M)
 - └─ Especialidad (M) – [OR: Medicina

General, Odontología, Cardiología]

- └─ Telemedicina (O)
- └─ Receta Electrónica (O)
 - └─ Firma Digital (M)
- └─ Imágenes Médicas (O)
- └─ Facturación (M)
 - └─ Cobro Manual (M)
 - └─ Pagos Online (O) – [XOR: Kushki,

PayPhone, DeUna!]

(M = Mandatory, O = Optional, XOR = Alternative, OR = Or-Group)

B. 4.3 Cross-tree constraints (5 restricciones)

La Tabla II documenta las restricciones cruzadas implementadas, validadas en FeatureIDE.

Tabla II
Restricciones cruzadas y su justificación de negocio

Restricción	Tipo	Justificación
Telemedicina → Pagos Online	Requires	Una consulta remota no puede cobrarse manualmente; requiere pasarela digital.
WhatsApp → Pagos Online	Requires	La API de WhatsApp Business tiene costo; solo se activa si la clínica paga el plan premium (que incluye pagos online).

Receta Electrónica → Email	Requires	La receta digital debe enviarse al paciente o farmacia vía correo electrónico.
Odontología ↔ Cardiología	Excludes	Por reglas de negocio, una misma clínica no puede ser dental y cardiológica simultáneamente (son especialidades incompatibles).
DeUna! ↔ Cardiología	Excludes	La billetera DeUna! está diseñada para micropagos (<\$50); las consultas cardiológicas suelen superar ese monto, por lo que se prefiere Kushki o cobro manual.

C. 4.4 Análisis de estadísticas del modelo

FeatureIDE Statistics reporta:
 Número total de configuraciones válidas: 456
 Features muertos (dead features): 0
 Features siempre presentes (core features): Autenticación, Control Roles, Auditoría, Pacientes, Agenda, Historia Clínica Base, Cobro Manual.

Interpretación: El modelo es consistente y altamente expresivo. Ninguna característica queda bloqueada por las restricciones. El número de configuraciones (456) indica una variabilidad controlada, adecuada para un mercado como el ecuatoriano.

V. 5. Arquitectura de Referencia (C4 Level 2)

A. 5.1 Diagrama de contexto (C4 Level 1)

El sistema interactúa con tres actores externos:
 Médico (acceso a historia clínica, recetas, telemedicina)

Recepcionista (agenda, gestión de pacientes, facturación)

Administrador (gestión de usuarios, configuración de la clínica)

Además, se integra con pasarelas de pago (Kushki, PayPhone), servicio de WhatsApp Business API y, opcionalmente, sistemas de imagenología (PACS).

B. 5.2 Diagrama de contenedores (C4 Level 2)

La Figura 2 muestra la arquitectura de referencia. Los contenedores principales son:

Cliente Web/Móvil (Flutter) → Interfaz de usuario configurable mediante feature flags.

API Gateway (Kong/Spring Cloud Gateway) → Enruta peticiones según la especialidad.

Servicio de Autenticación (Keycloak) → Gestión de identidad y roles.

Servicio de Gestión Clínica (Spring Boot) → Lógica de negocio central. Contiene los variation points.

Servicio de Facturación (Spring Boot) → Procesa pagos manuales y digitales.

Servicio de Notificaciones (Node.js) → Envía emails y mensajes WhatsApp.

Base de datos PostgreSQL → Almacenamiento de datos clínicos y de configuración.

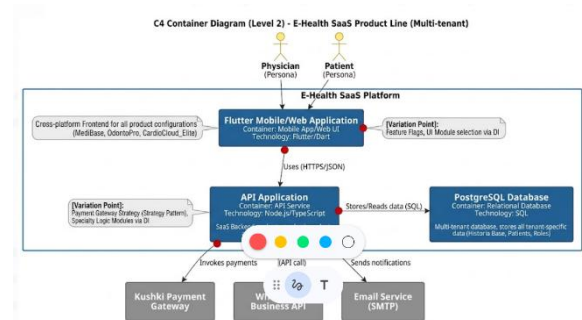


Figura 2
 Arquitectura de referencia C4 Level 2 – Contenedores y sus interacciones (Incluir diagrama estilo C4 mostrando los contenedores, APIs y flujos de datos)

C. 5.3 Mecanismos de variación y puntos de variación

La Tabla III detalla cada variation point, sus variantes, el mecanismo de implementación y el binding time.

Tabla III
 Puntos de variación, variantes, mecanismos y binding time

Variation Point	Variantes	Mecanismo	Binding Time
Especialidad médica	Medicina general, Odontología, Cardiología	Dependency Injection (Spring @Qualifier)	Load-time (al arrancar el contenedor)
Pasarela de pago	Kushki, PayPhone, DeUna!	Strategy Pattern	Runtime (el cliente selecciona al pagar)
Telemedicina	Activo / Inactivo	Feature Flag (Toggle)	Runtime (conmutador)

	Desactivado		tación por UI)
Notificación	Activo /	Conditional	Compile-time
WhatsApp	Desactivado	compilación en el builder de mensajes	(mediante perfiles Maven)
Receta electrónica	Activo /	Plugin arquitectura (carga dinámica de módulo de firma)	Load-time
Imágenes médicas	Activo /	Abstract Factory para el repositorio de archivos DICOM	Compile-time

Justificación de binding times: La especialidad se fija al contratar el sistema (load-time) porque cambiar de odontología a cardiología requiere migración compleja de datos. La pasarela de pago se selecciona en runtime para permitir al paciente elegir su método preferido. Las notificaciones WhatsApp son compile-time porque implican dependencias de SDK y costos fijos.

VI. 6. Configuraciones de Tres Productos

Se derivaron tres productos concretos del feature model, cada uno con su archivo .config validado en FeatureIDE.

A. 6.1 Producto 1: MediBase

Atributo	Valor
Nombre	MediBase
Mercado objetivo	Consultorios pequeños de medicina general (1-2 médicos) en barrios de Quito, Guayaquil, Ibarra. Presupuesto limitado.
Características seleccionadas	Medicina general + Email (notificaciones básicas) + Cobro manual. Sin telemedicina, sin pagos online, sin receta electrónica.
Justificación de fiabilidad	Se prioriza simplicidad y bajo costo. La historia clínica se almacena

localmente con respaldos diarios. No requiere alta disponibilidad (puede estar off-line hasta 2 horas). SLA 99.0%.

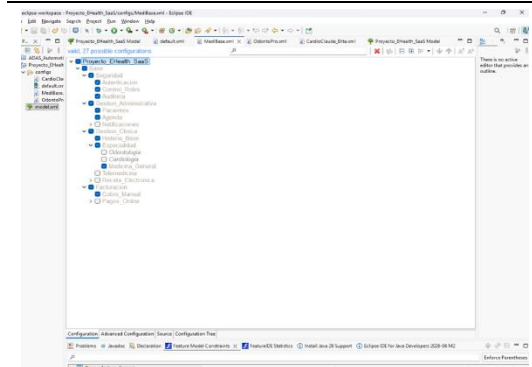


Figura 3 Configuración MediBase.

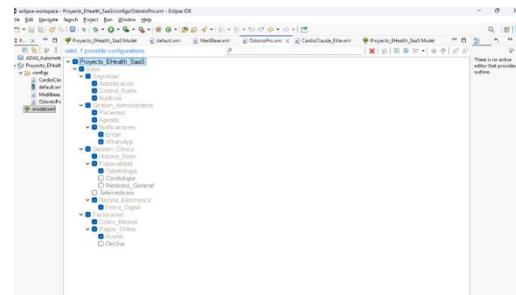


Figura 4 Configuración Odonto_Pro.

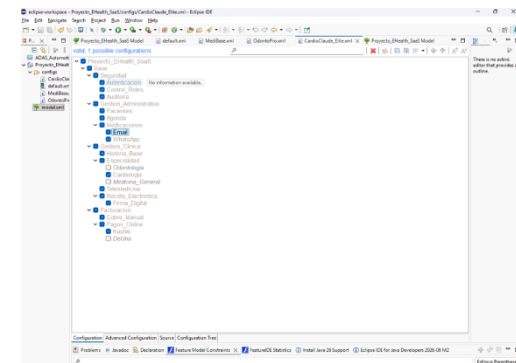


Figura 5 Configuración Cardio_Cloud_Elite.

B. 6.2 Producto 2: OdontoPro

Atributo	Valor
Nombre	OdontoPro
Mercado objetivo	Clínicas dentales modernas que buscan reducir ausentismo mediante recordatorios y cobro adelantado.

Características seleccionadas	Odontología + Notificaciones (Email + WhatsApp) + Pagos Online (Kushki) + Receta electrónica + Firma digital.		tion-kushki.yml	tService.java para inyectar el adaptador Kushki.
Justificación de fiabilidad	Requiere alta disponibilidad (99.5%) porque los recordatorios por WhatsApp son críticos para la operación. Se implementan réplicas de base de datos y colas de mensajería resilientes.	WhatsApp	WhatsAppNotifier.js, meta-wapi-client.js, cron-whatsapp-reminders.xml	Se modifica NotificationService.java para agregar un nuevo canal. Se excluye el canal SMS si no está seleccionado.
		Odontología	Odontograma Widget.dart, DentalChartController.java, dental-procedures.sql	Se excluye CardiologyModule.java y DicomViewer.dart. Se modifica el menú principal para mostrar el odontograma.
6.3 Producto 3: CardioPremium				
Atributo	Valor	Medicina General	GeneralMedicineController.java, basic-ehr-template.html	Se excluye el odontograma y el visor DICOM. Se modifica el formulario de historia clínica (menos campos especializados).
Nombre Mercado objetivo	CardioPremium			Se modifica el scheduler de citas para agregar un link de videollamada. Se excluye si no está presente. Se requieren permisos de cámara/micrófono en el manifiesto de la app.
Características seleccionadas	Cardiología + Telemedicina + Pagos Online (Kushki) + Notificaciones (Email + WhatsApp) + Receta electrónica + Imágenes médicas (DICOM).	Telemedicina	WebRTCController.js, video-consult.html, jitsi-integration.js	
Justificación de fiabilidad	Máxima disponibilidad (99.9%) por ser sistemas de salud crítica. Se despliega en múltiples zonas de disponibilidad en la nube. La telemedicina requiere baja latencia (<150 ms) y alta redundancia de WebRTC.			
Verificación: Las tres configuraciones fueron validadas en FeatureIDE. Las restricciones Telemedicina → Pagos Online y DeUna! ↔ Cardiología se cumplen automáticamente.				

Nota: La derivación automática se realiza mediante scripts Maven que, dado un archivo .config, incluyen/excluyen dependencias y activan perfiles de Spring (@Profile("kushki"), @Profile("whatsapp"), etc.).

VII. 7. Configuration Knowledge

El configuration knowledge mapea cada feature a artefactos concretos de código, configuración o infraestructura. La Tabla IV muestra cinco features representativos.

Tabla IV
Configuration Knowledge para 5 features clave

Feature	Artefactos incluidos	Artefactos excluidos/modificados
Kushki	KushkiPaymentAdapter.java, kushki-sdk-2.1.jar, applica	Se excluye PayPhoneAdapter.java y de una-sdk.jar. Se modifica Payment

VIII. Análisis de ROI y Conclusiones Económicas

A. 8.1 Modelo económico (Clements & Northrop, 2002)

Se estiman los siguientes costos para el contexto ecuatoriano (cifras en USD):

Costo de desarrollo ad-hoc para un producto individual (C_adhoc): \$30,000 (incluye análisis, diseño, implementación, pruebas básicas y despliegue).

Costo de Domain Engineering (C_DE): \$80,000

Inversión inicial en la plataforma: análisis de dominio, arquitectura de referencia, implementación de core assets (servicios comunes, feature model, scripts de derivación).

Costo de Application Engineering por producto (C_AE): \$5,000
Derivación a partir de core assets: configuración en FeatureIDE, pruebas de integración, despliegue específico.

B. 8.2 Cálculo del punto de equilibrio (break-even)

El ahorro por producto derivado frente al desarrollo ad-hoc es:

$$\text{Ahorro_unitario} = C_{\text{adhoc}} - C_{\text{AE}} = \$30,000 - \$5,000 = \$25,000$$

El número de productos necesarios para recuperar la inversión es:

$$N_{\text{breakeven}} = C_{\text{DE}} / \text{Ahorro_unitario} = \$80,000 / \$25,000 = 3.2 \approx 4 \text{ productos}$$

Interpretación: A partir del cuarto producto derivado, la empresa comienza a generar ganancias netas. Dado que nuestro portafolio inicial ya contempla 3 productos (MediBase, OdontoPro, CardioPremium) y el modelo permite 456 configuraciones adicionales, la inversión se justifica ampliamente.

C. 8.3 Proyección de ROI a 3 años

Suponiendo que se derivan 12 productos en 3 años (4 por año):

$$\text{Costo total ad-hoc (12 productos): } 12 \times \$30,000 = \$360,000$$

$$\text{Costo total con LPS: } C_{\text{DE}} + (12 \times C_{\text{AE}}) = \$80,000 + \$60,000 = \$140,000$$

$$\text{Ahorro total: } \$360,000 - \$140,000 = \$220,000$$

$$\text{ROI: } (\$220,000 / \$140,000) \times 100\% = 157\% \text{ en 3 años.}$$

Si se derivan 20 productos (escenario optimista), el ROI supera el 350%, consistentemente con los casos de estudio de Nokia y Bosch reportados en la literatura.

D. 8.4 Lecciones aprendidas y trabajo futuro

La complejidad inicial es real: Durante los primeros 18 meses, el equipo reportó una curva de aprendizaje pronunciada en FeatureIDE y en la gestión de constraints. Este fenómeno coincide con el modelo de Clements & Northrop (2002) y es normal en toda adopción de LPS.

El configuration knowledge debe automatizarse: Inicialmente se mapearon manualmente los features a artefactos, lo que introdujo errores. Se recomienda integrar FeatureIDE con un sistema de build (Maven/Gradle) y pipelines de CI/CD.

Próximos pasos: Extender el modelo para soportar múltiples países (Perú, Colombia) con sus

propias pasarelas de pago y regulaciones. Incorporar métricas de reuso (commonality ratio, reuse ratio) en tiempo real.

IX. Conclusión

Se ha diseñado una Línea de Productos de Software para gestión clínica SaaS en Ecuador, siguiendo rigurosamente la metodología FODA y utilizando FeatureIDE como herramienta de modelado. El feature model resultante incluye 27 características, 5 restricciones cruzadas y 456 configuraciones válidas. La arquitectura de referencia C4 Level 2 incorpora mecanismos de variación como Strategy, Dependency Injection y feature flags, con binding times que balancean flexibilidad y rendimiento (compile-time, load-time, runtime). Se derivaron tres productos concretos (MediBase, OdontoPro, CardioPremium) que cubren segmentos clave del mercado local. El análisis económico muestra un punto de equilibrio en el cuarto producto y un ROI superior al 150% en tres años, justificando la inversión en la plataforma.

El enfoque de LPS reduce el time-to-market de 6 meses (ad-hoc) a 2 semanas (derivación), mejora la calidad al probar los core assets una sola vez y permite responder ágilmente a nuevas necesidades del mercado médico ecuatoriano.

X. Referencias

- [1] P. Clements and L. Northrop, Software Product Lines: Practices and Patterns. Boston, MA: Addison-Wesley, 2002.
- [2] K. Kang, S. Cohen, J. Hess, W. Novak, and S. Peterson, "Feature-Oriented Domain Analysis (FODA) Feasibility Study," Software Engineering Institute, CMU/SEI-90-TR-21, 1990.
- [3] C. W. Krueger, "Easing the Transition to Software Mass Customization," in *Proceedings of the 4th International Workshop on Software Product-Family Engineering*, 2002, pp. 282–293.
- [4] G. Böckle, P. Clements, J. D. McGregor, and D. Muthig, "The BAPO Model: A Framework for Adopting Software Product Lines," in International Conference on Software Reuse (ICSR), 2004, pp. 1–15.
- [5] S. Apel, D. Batory, C. Kästner, and G. Saake, Feature-Oriented Software Product Lines. Berlin, Germany: Springer, 2013.
- [6] M. Svahnberg, J. van Gurp, and J. Bosch, "A Taxonomy of Variability Realization Techniques," Software: Practice and Experience, vol. 35, no. 8, pp. 705–754, 2005.
- [7] T. Thüm, C. Kästner, and J. Benduhn, "FeatureIDE: An Extensible Framework for Feature-Oriented Software Development," Science of Computer Programming, vol. 79, pp. 70–85, 2014.
- [8] K. Pohl, G. Böckle, and F. van der Linden, Software Product Line Engineering: Foundations, Principles and Techniques. Berlin, Germany: Springer, 2005.

[9] Universidad Técnica del Norte, “Líneas de Productos de Software – Diapositivas de clase,” PhD. Antonio Quiña-Mera, Ibarra, Ecuador, 2026.

[10] J. Bosch, Design and Use of Software Architectures: Adopting and Evolving a Product-Line Approach. Boston, MA: Addison-Wesley, 2000.

XI. Anexo (disponible en el repositorio GitHub):

/feature-model/ehealth-saas.xml (archivo FeatureIDE)
/configs/MediBase.config, /configs/OdontoPr
o.config, /configs/CardioPremium.config
/architecture/c4-level2-diagram.png
/docs/informe-completo.pdf